

Important Note

You must write your code independently. Do not copy the code from any source. To receive full credit, your code must be well-documented with comments. Your script **must** be uploaded as either a .py or .ipynb file to Brightspace.

Project Requirements

Due Date

Sunday, 11:59 PM EST, November 2nd, 2025

- 1-2 page report, includes:
 - work done
 - results
 - discussion of results
 - 500 words of text max (excluding figure captions)
- One ZIP file containing commented, executable code

1 Mandelbrot Set (30 Points)

Background: Consider the following iteration

$$z_{n+1} = z_n^2 + c, \quad z_i, c \in \mathbb{C} \quad (1.1)$$

Suppose $z_0 = 0$. Depending on our choice of c , the iteration 1.1 will either be bounded or diverge to infinity. The set

$$M = \{c : \text{the iteration using } c \text{ is bounded}\} \quad (1.2)$$

is known as the *Mandelbrot set*. That is, a complex number c is in the Mandelbrot set M if, using c in Eq. 1.1, $|z_i| < \infty$ for all $i > 0$. A plot of the Mandelbrot set is shown in Fig. 1.

Task: Write a Python script called `mandelbrot.py` which computes the Mandelbrot set over the range $[-2, 1] \times [-1.5, 1.5]$ using a threshold of 50. Your script should return an image of the Mandelbrot fractal similar to Fig. 1, and this image should be included in your final report.

Hints:

- For a complex number $z = x + iy$, where $x, y \in \mathbb{R}$.

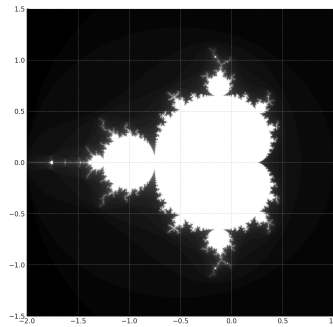


Figure 1: Mandelbrot set

- For a complex number z , $|z| = \sqrt{x^2 + y^2}$. When used on a complex number, the `abs()` method in Python will do this automatically.
- You may approximate $|z| < \infty$ by using $|z| < \text{threshold}$, where `threshold` is a parameter. Similarly, we cannot iterate until infinity so choose a max number of iterations, say `N_max`.
- Instead of computing c one at a time, you may define c to be the grid of points $[-2, 1] \times [-1.5, 1.5]$. For example, to construct a 50×50 grid, you may use the `numpy.mgrid` function:

```

1  x, y = numpy.mgrid[-2:1:50j, -1.5:1.5:50j]
2  c = x + 1j * y
3

```

The resolution of your image may be improved by increasing the number of grid points.

- To avoid overflow of floating point arithmetic, modify the iteration to update z_{ij} only if $|z_{ij}| < 100^{100}$ (or more efficiently, if $|z_{ij}| < \text{threshold}$).
- Once your Mandelbrot iteration is completed, create a `mask` from z that will indicate which points are in the Mandelbrot set (i.e., $|z_{ij}| < \text{threshold}$)
- To plot the fractal and save it as an image, you may use

```

1  plt.imshow(mask.T, extent=[-2, 1, -1.5, 1.5])
2  plt.gray()
3  plt.savefig('mandelbrot.png')
4  plt.show()
5

```

2 Markov Chain (30 Points)

Background: Consider a Markov chain transition matrix P on a set of n states, where P_{ij} is the probability ($0 \leq P_{ij} \leq 1$) to go from state i to state j , and each row is normalized so that:

$$\sum_{j=1}^n P_{ij} = 1 \quad (2.1)$$

Let p be a size- n vector composed of the probability distribution over the n states, where

$$\sum_{j=1}^n p_j = 1. \quad (2.2)$$

The transition rule corresponding to the transition matrix P is $\hat{p} = P^T p$, where $\hat{p}$ is a new probability distribution and

$$\sum_{j=1}^n \hat{p}_j = 1. \quad (2.3)$$

Task: Write a Python script named `markov_chain.py` that works with 5 states. In particular,

1. Construct a random 5×5 matrix P and normalize each row so that

$$\sum_{j=1}^5 p_{ij} = 1.$$

2. Construct a random size-5 vector p and normalize it.

$$\sum_{j=1}^5 p_j = 1.$$

Apply the transition rule 50 times to obtain p_{50} .

3. Compute the eigenvector v of P^T corresponding to the eigenvalue 1, (and numerically, the eigenvalue closest to 1), and then scale the eigenvector so that

$$\sum_{j=1}^5 v_j = 1.$$

This scaled eigenvector is known as the *stationary distribution*.

4. Compute the component-wise difference between p_{50} and the stationary distribution. Do they match within 10^{-5} ?

Hints: you may find the following functions useful in your implementation:

- `np.random.rand`
- `np.dot`
- `np.linalg.eig`.

3 Taylor Series Approximation (40 Points)

Background: Recall for $f \in C^\infty(\mathbb{R})$ (i.e., a function f that is infinitely differentiable), its *Taylor series* about a point $c \in \mathbb{R}$ is the following infinite series:

$$\sum_{n=0}^{\infty} \frac{f^{(n)}(c)}{n!} (x-c)^n = f(c) + f'(c)(x-c) + \frac{f''(c)}{2}(x-c)^2 + \frac{f'''(c)}{3!}(x-c)^3 + \dots$$

Furthermore, f is said to be analytic in an interval I , if at any point $c \in I$ it agrees with its Taylor series. In other words, f is said to be analytic on I if and only if for any $c \in I$:

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(c)}{n!} (x-c)^n, \quad \forall x \in I. \quad (3.1)$$

Notice, we may use Eq. 3.1 to approximate an analytic function f using a truncated Taylor series. That is, for $m \in \mathbb{N}$ we have

$$f(x) \approx \sum_{n=0}^m \frac{f^{(n)}(c)}{n!} (x-c)^n. \quad (3.2)$$

The greater m is, the better our approximation will be.

Task: Write a Python script named `taylor.py` that is able to approximate any analytic function. In particular::

1. **(20 points)** Write a function that takes in the following 5 inputs:
 - (a) `func`, the function you would like to approximate
 - (b) `start`, the beginning of your interval
 - (c) `end`, the end of your interval
 - (d) `degree`, the last integer where you will be truncating your series (i.e. m in Eq. 3.2)
 - (e) `fixed_c`, the point you will be expanding around (i.e., c in Eq. 3.2)

and outputs a NumPy array of points approximating f in the interval `(start, end)`.

2. **(10 points)** Run the function you wrote above in Q3.1 using the following parameters:
 - (a) $f(x) = x \sin^2(x) + \cos(x)$
 - (b) Numerical domain: $[-10, 10]$ with 100 points
 - (c) Use a total of 100 terms in your series (i.e., $m = 99$ in Eq. 3.2)
 - (d) Expand around the point $c = 0$ (i.e. $c = 0$ in Eq. 3.2)

and plot both the function and the approximation you computed. Your plot should include a legend and labelled axes. See Fig. 2 for an example.

Factorial (10 points): Write a function that, in addition to all the inputs of Q3.1, takes in the following three inputs:

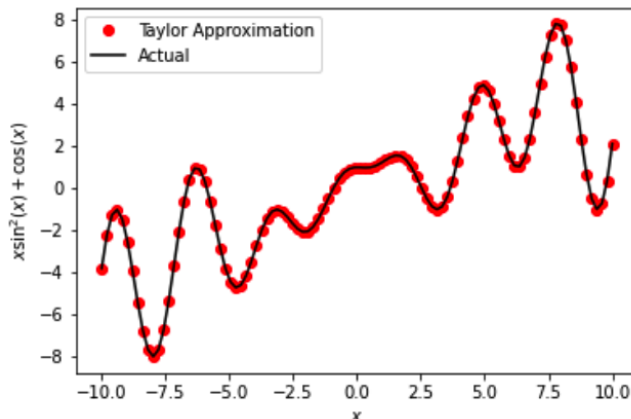


Figure 2: Example Taylor series approximation plot.

- `initial_degree`
- `final_degree`
- `degree_step`

and runs your function from Q3.1 for various cut off values (i.e. m in Eq. 3.2), starting from initial degree until degree step, with step size degree step. For example, suppose you gave the following inputs into your function:

- `initial_degree = 50`
- `final_degree = 100`
- `degree_step = 10`

then your function from Q3.1 will compute the Taylor series approximation first for $m = 50$, and then for $m = 60, 70, \dots, 100$. In each iteration you should:

1. Compute the sum of the absolute value of the difference between the true value and the approximation. In other words, if your domain consists of n points then compute:

$$\sum_{i=0}^{n-1} |f(x_i) - \hat{f}(x_i)| \quad (3.3)$$

where $\hat{f}$ denotes the approximation.

2. Time how long it takes to compute the approximation.

The values you obtain from Q3.3a and Q3.3b should then be stored as separate columns in a **Pandas** dataframe, with each row corresponding to some cutoff value m . Once you have iterated through all cutoff values m , write your **Pandas** dataframe to a **csv** file titled `taylor_values.csv`, and submit this **csv** file as part of your report.

HINTS: You may find the following libraries and functions useful in your implementation:

- To compute and evaluate the function and its derivatives you may use the `SymPy` library
- `numpy.math.factorial()`